AgroManejo

Especificações técnicas para o desenvolvimento

Novembro de 2026

Especificação 01 — Protocolo de sincronização

Complementa a seção 7.4 da proposta técnica (`entrega-basica/AgroManejo-Entrega-Basica.md`). Define o comportamento necessário para implementar a sincronização entre o SQLite do aparelho e o PostgreSQL (Supabase) sem perda nem duplicação de dados.

1. Princípios

1. **O aparelho é a fonte de trabalho; o servidor é a fonte de verdade compartilhada.** O produtor lê e grava sempre no SQLite. O servidor decide se uma alteração é aceita.
2. **O relógio do aparelho nunca decide nada.** Ordem, versão e cursor vêm do servidor.
3. **Toda operação pode ser repetida sem efeito colateral** (idempotência).
4. **Nenhuma alteração é sobrescrita sem que o produtor a tenha visto.**

2. Unidades de sincronização

Unidade	Conteúdo	Observação
Unidade de manejo	Dados da unidade **e** todos os seus vértices	Bloco único: o contorno inteiro é aceito ou recusado junto. Os vértices nunca são sincronizados isoladamente.
Atividade	Todos os campos da atividade	Independente da unidade; referencia a unidade pelo identificador.

3. Estado de cada registro no aparelho

Situação	Significado	Pode ser editado?
SINCRONIZADO	Igual à versão do servidor.	Sim (passa a PENDENTE)
PENDENTE	Alterado no aparelho e ainda não aceito pelo servidor.	Sim
CONFLITO	O servidor tem uma versão mais nova feita em outro aparelho.	**Não**, até o produtor resolver (UC10)
REJEITADO	O servidor recusou por regra de validação. O motivo fica registrado.	Sim (a correção volta a PENDENTE)

Cada registro guarda também:

- **versão-base:** a versão do servidor sobre a qual a edição foi feita (nula em registros novos);
- **identificador da operação pendente:** um UUID gerado quando o registro passa a PENDENTE e regenerado a cada nova edição local. Garante a idempotência.

O bloqueio de edição em CONFLITO evita conflitos sobre conflitos, que o produtor não conseguiria entender.

4. Ciclo de sincronização

Gatilhos:

- conexão restabelecida;
- aplicativo trazido para o primeiro plano;
- 5 segundos após a última gravação local, com conexão;
- fim do login.

Só um ciclo roda por vez; um gatilho durante um ciclo agenda o próximo.

Ordem:

1. Renovar a sessão, se o token estiver vencido.
2. **Envio** (seção 5), em lotes de até 100 registros PENDENTE.
3. **Recebimento** (seção 6), em páginas de até 500 alterações, até não haver mais.
4. Gravar o horário do último ciclo bem-sucedido, informado pelo servidor.

O envio vem antes do recebimento para que o conflito seja detectado pelo servidor, com versão, e não por sobrescrita local.

5. Envio de alterações

O aplicativo chama **uma função do banco** (RPC do Supabase), não as tabelas diretamente. Uma função do PostgreSQL executa dentro de uma transação, então a gravação da unidade com todos os vértices é atômica.

Conteúdo do lote, para cada registro:

- identificador do registro;
- identificador da operação;
- tipo (unidade ou atividade);
- versão-base;
- dados completos (para a unidade, inclui a lista ordenada de vértices).

O aplicativo também envia a sua **versão**.

Processamento no servidor, registro a registro, cada um isolado dos demais, para que um conflito não bloqueie o lote:

1. Se o identificador da operação já foi processado, devolver o mesmo resultado anterior (idempotência).
2. Conferir que o registro pertence ao produtor autenticado.
3. Validar:
    - polígono válido, reconstruído a partir dos vértices com as funções do PostGIS;
    - no mínimo 3 vértices;
    - campos obrigatórios presentes;
    - valores monetários não negativos.
4. Comparar a versão-base com a versão atual:

    - registro inexistente e versão-base nula → **inserir** com versão 1;
    - versões iguais → **gravar** e incrementar a versão;
    - versões diferentes → **conflito**: nada é gravado.
5. Para unidades: recalcular a área com o PostGIS e gravar esse valor, que passa a ser a área oficial.
6. Registrar a alteração no registro de alterações (seção 6) e o identificador da operação como processado.

Resposta por registro:

Resultado	Dados devolvidos	Ação no aparelho
ACEITO	nova versão, área oficial (unidades)	SINCRONIZADO; versão-base = nova versão; atualizar a área
CONFLITO	versão atual do servidor	CONFLITO; manter a versão local intacta
REJEITADO	motivo da recusa	REJEITADO; mostrar o motivo ao produtor

Se a resposta se perder (queda de conexão), o reenvio com o mesmo identificador de operação devolve o mesmo resultado, sem gravar de novo.

6. Recebimento de alterações (cursor)

Problema a evitar: um contador simples pode pular registros. Uma transação que começou antes pode terminar depois e ganhar um número menor que um cursor já lido.

Mecanismo:

- O servidor mantém um **registro de alterações**: cada gravação aceita gera uma linha com número sequencial, produtor, tipo, identificador do registro, nova versão e o **identificador da transação** que a gravou.
- O recebimento devolve apenas as alterações:
    - do produtor autenticado,
    - com número maior que o cursor do aparelho,
    - **cujas transações já terminaram antes do início de qualquer transação ainda em andamento.** No PostgreSQL 13 ou superior, isso usa as funções `pg_current_xact_id()`, `pg_current_snapshot()` e `pg_snapshot_xmin()`.

  Alterações de transações ainda abertas ficam para o ciclo seguinte, em vez de serem puladas.
- O aparelho guarda o maior número recebido como novo cursor, **só depois** de gravar a página localmente.

Aplicação no aparelho:

Situação local do registro	O que acontece com a alteração recebida
Não existe ou SINCRONIZADO	Gravada; situação SINCRONIZADO
PENDENTE	**Não aplicada.** O próximo envio detectará o conflito pela versão.
CONFLITO	Não aplicada

Situação local do registro	O que acontece com a alteração recebida
Mesma versão já presente	Ignorada (repetição)

O primeiro acesso em um aparelho é um recebimento com cursor zero, paginado, exibindo o progresso.

7. Resolução de conflito (UC10)

Mantém a regra já aprovada:

1. O aplicativo busca a versão atual do servidor e a exibe ao lado da local.
2. Depois da escolha do produtor, consulta de novo a versão do servidor.
3. Se for mantida a versão local, envia com versão-base igual à versão exibida. O servidor só aceita se ela ainda for a atual; se não for, o conflito é reapresentado.
4. Se for adotada a versão do servidor, ela substitui a local, que passa a SINCRONIZADO.

8. Tratamento de erros

Situação	Comportamento
Sem conexão ou tempo esgotado	Nova tentativa automática com intervalos crescentes: 5 s, 15 s, 45 s, 2 min, até no máximo 10 min entre tentativas.
Sessão vencida	Renovar a sessão e repetir. Se a renovação falhar, pedir novo login **sem apagar** pendências.
Acesso negado (regra de acesso por linha)	Registro REJEITADO e erro registrado para diagnóstico. Indica defeito, não uso normal.
Versão do aplicativo incompatível	Interromper a sincronização e pedir atualização do aplicativo. Os dados locais continuam utilizáveis offline.
Erro inesperado do servidor	Tratar como falha de rede (nova tentativa) e registrar para diagnóstico.

9. Compatibilidade de versões

- O servidor mantém a **versão mínima aceita do aplicativo**. Lotes de versões mais antigas são recusados com um código específico.
- Toda mudança de estrutura no servidor deve continuar aceitando a versão mínima vigente. Campos novos são opcionais até que todas as versões em uso os enviem.

10. Registro para diagnóstico

O aplicativo registra localmente, e envia quando houver conexão:

- falhas de sincronização (tipo, código, registro afetado);
- conflitos;
- rejeições.

Nenhum dado pessoal ou financeiro vai nesse registro, só identificadores e códigos.

11. Pontos a confirmar na implementação

- Comportamento transacional exato das chamadas RPC via API do Supabase (PostgREST). A página de documentação correspondente não pôde ser consultada nesta revisão.
- Tamanhos de lote e intervalos (100 / 500 / 5 s): valores iniciais, a ajustar com medições no protótipo.

Especificação 02 — Mapa offline: provedor e estratégia de pacotes

Complementa as seções 1.4 e 7.2 da proposta técnica. Objetivo: garantir que o mapa da propriedade esteja **sempre** disponível no aparelho, sem violar os termos de uso das fontes de imagem.

1. Requisito que orienta a decisão

A imagem de fundo precisa ficar armazenada no aparelho **por tempo indeterminado** e ser baixada **antecipadamente** para a região da propriedade, antes de o produtor visitá-la. Isso elimina qualquer fonte que só permita cache temporário ou proíba download antecipado.

2. Avaliação dos provedores (termos consultados em nov/2026)

Fonte	O que os termos dizem	Atende?
Servidores de mapa do OpenStreetMap (tile.openstreetmap.org)	"Offline use is not permitted on tile.openstreetmap.org." O download antecipado de regiões é proibido. A política recomenda servidor próprio ou provedor que permita uso offline.	**Não**
Mapbox	Cache no aparelho permitido, mas "limited to thirty (30) days on the same device", dentro dos limites dos SDKs da própria Mapbox, e populado diretamente pela API.	**Não** (cache expira; sem download antecipado garantido)
MapTiler Cloud	Só cache pessoal temporário. "It is prohibited to batch or excessive bulk download of map tiles."	**Não**
MapTiler (dados para servidor próprio)	Plano padrão: um aplicativo interno, até 500 usuários ativos por mês, sem uso B2B/B2C. Uso para consumidores exige contrato personalizado.	Só com **contrato**
Esri World Imagery	Uso offline documentado apenas pelos SDKs nativos da Esri, com conta ArcGIS. O MapLibre não aparece como opção para uso offline.	**Não** com MapLibre
Google Maps Platform	Termos não puderam ser lidos (a página exige JavaScript).	**Não verificado**; não recomendado sem leitura jurídica
EOX Sentinel-2 cloudless	Gratuito para uso **não comercial** (CC BY-NC-SA 4.0). Uso comercial com licença paga da EOX. Resolução de 10 m.	Sim para o **protótipo**; comercial com licença
Dados abertos processados pelo projeto (OpenStreetMap para ruas e rótulos; imagens Copernicus Sentinel-2)	Os dados podem ser processados e servidos pelo próprio projeto, cumprindo as regras de atribuição de cada licença.	**Sim**, com servidor próprio
Imagens do próprio produtor (drone)	O produtor é dono da imagem.	**Sim** (trabalho futuro)

3. Decisão recomendada

Servir os pacotes de mapa a partir de infraestrutura própria, com dados abertos, em duas camadas:

1. **Camada de referência (vetorial):** estradas, rios, limites e nomes, gerados a partir de dados do OpenStreetMap para a área de atuação. São leves, nítidos em qualquer zoom e livres para uso offline quando servidos pelo próprio projeto, com a atribuição "© colaboradores do OpenStreetMap".
2. **Camada de imagem (satélite):**
    - **protótipo:** mosaico Sentinel-2 cloudless da EOX, uso não comercial, com a atribuição exigida;
    - **produção:** mosaico Sentinel-2 processado pelo projeto a partir dos dados Copernicus, sem licença paga, ou uma fonte comercial de maior resolução contratada com cláusula expressa de uso offline.

Hospedagem: os arquivos de blocos ficam no armazenamento de arquivos do próprio Supabase ou em uma CDN, em formato que o MapLibre lê diretamente. O `OfflineManager` do MapLibre baixa deles os pacotes.

Limitação a declarar: imagens de 10 m mostram talhões, estradas e mudanças de cultura, mas não detalhes finos como cercas ou carreadores estreitos. A precisão do contorno vem do GPS e das coordenadas, não da imagem. Para detalhe fino, o caminho é a imagem de drone (trabalho futuro).

4. Estratégia de pacotes

Um pacote por grupo de unidades próximas, e não um retângulo único para toda a propriedade.

- **Agrupamento:** unidades cujos contornos estejam a menos de 2 km entre si formam um grupo.
- **Região do pacote:** o retângulo que envolve o grupo, mais uma margem de 500 m.
- **Níveis de zoom:**
    - imagem de satélite de 10 m: do zoom 10 (visão geral) ao **15**. Acima disso a imagem não ganha detalhe; o mapa amplia o último nível disponível;
    - camada vetorial: do zoom 10 ao 14, ampliada acima disso sem perda de nitidez.
- **Por que não um único retângulo:** talhões distantes (por exemplo, 15 km) gerariam uma região enorme sem interesse, multiplicando espaço e dados móveis.

Ordem de grandeza do tamanho. Na latitude do Rio Grande do Sul (cerca de 30° S), um bloco no zoom 15 cobre cerca de 1 km de lado. Uma propriedade de 5 km × 5 km com margem precisa de algumas dezenas de blocos nesse nível, e de menos nos níveis abaixo. A conta é mostrada para dar a escala; o tamanho real em megabytes depende do formato e da compressão e deve ser medido no protótipo.

Gestão dos pacotes no aparelho:

Evento	Ação
Unidade criada ou contorno alterado	Recalcular os grupos; baixar só as regiões novas ou ampliadas.
Antes de cada download	Estimar o número de blocos e o tamanho; conferir o espaço livre; avisar o produtor se não houver espaço.

Evento	Ação
Limite de blocos	Configurar o limite de blocos do `OfflineManager` (`setTileCountLimit`) com folga para a maior propriedade esperada, respeitando os termos da fonte.
Pacote sem unidades	Remover após a sincronização confirmar que nenhuma unidade o usa.
Atualização da imagem	Novo mosaico publicado pelo servidor; atualizar com Wi-Fi ou quando o produtor pedir.

5. Atribuição obrigatória na tela do mapa

Exibir, de forma discreta e acessível, por exemplo em um ícone “i”:

- “© colaboradores do OpenStreetMap” (camada vetorial);
- o texto exigido pela fonte de imagem. Para o EOX cloudless: “EOxCloudless by EOX IT Services GmbH (Contains modified Copernicus Sentinel data [ano])”.

6. Decisões pendentes

1. Confirmar a área de atuação inicial (estado ou região) para gerar a camada vetorial.
2. Para produção: processar o próprio mosaico Sentinel-2 ou contratar imagem comercial de maior resolução com cláusula de uso offline.
3. Se o Google ou outro provedor comercial for considerado, ler os termos completos antes de qualquer uso offline.

Fontes consultadas

- OSM Foundation, *Tile Usage Policy*: https://operations.osmfoundation.org/policies/tiles/
- Mapbox, *Product Terms* (20/10/2026), seção 2.8.1.
- MapTiler, *Cloud Terms*: https://www.maptiler.com/terms/cloud/ — e *Server & Data Terms*: https://www.maptiler.com/terms/server-data/
- Esri, *Offline mapping apps*: https://developers.arcgis.com/documentation/offline-mapping-apps/
- EOX, *EOxCloudless license*: https://cloudless.eox.at/documentation/license
- MapLibre React Native, *OfflineManager*: https://maplibre.org/maplibre-react-native/docs/modules/offline-manager

Especificação 03 — Ajustes do modelo de dados

Complementa a seção 6 da proposta técnica. Lista as mudanças necessárias no modelo antes da criação das tabelas, para dar suporte à sincronização (especificação 01), ao mapa offline (especificação 02) e à segurança.

1. Mudanças nas tabelas existentes

Tabela	Mudança	Motivo
UNIDADE_MANEJO, VERTICE, ATIVIDADE	Acrescentar `produtor_id` em VERTICE e ATIVIDADE (UNIDADE_MANEJO já tem).	As regras de acesso por linha e as consultas de sincronização filtram direto pelo dono, sem consultar outra tabela.
Todas	Acrescentar `criado_em`, definido pelo servidor.	Auditoria e diagnóstico.
UNIDADE_MANEJO, ATIVIDADE	Manter `versao` e `atualizado_em`, ambos definidos **só** pelo servidor.	Controle de concorrência (especificação 01).
UNIDADE_MANEJO	`area_hectares` passa a ser **recalculada pelo servidor** (PostGIS) a cada gravação aceita.	Define a fonte da verdade da área. O valor do aparelho é provisório até a sincronização.
VERTICE	Sem `versao` própria; os vértices são regravados em bloco com a unidade.	O contorno é um bloco único.
ATIVIDADE	Custos e receita em reais com 2 casas decimais no servidor; **centavos como número inteiro** no aparelho.	O SQLite não tem tipo decimal exato; somas em ponto flutuante geram diferenças de centavos.
VERTICE	Latitude e longitude com 6 casas decimais.	Precisão de cerca de 0,1 m, suficiente para GPS de celular.

2. Regras de integridade (servidor)

- Valores monetários, tempo estimado e área: **nunca negativos**.
- `tipo` da atividade restrito a: AMOSTRAGEM_MONITORAMENTO, PLANTIO, COLHEITA, ADUBACAO, OUTRA.
- `avaliacao`: nula, POSITIVA ou NEGATIVA.
- `origem` do vértice: MANUAL, GEOLOCALIZACAO ou MAPA.
- Par (`unidade_id`, `ordem`) único.
- Latitude entre −90 e 90; longitude entre −180 e 180.
- Polígono válido e com no mínimo 3 vértices, verificado na função de envio (especificação 01, seção 5).

3. Novas tabelas no servidor

Tabela	Conteúdo	Uso
REGISTRO_ALTERACAO	número sequencial, `produtor_id`, tipo do registro, identificador do registro, nova versão, identificador da transação, data	Cursor de recebimento sem pular registros.
OPERACAO_PROCESSADA	identificador da operação, `produtor_id`, resultado, data	Idempotência dos envios. Pode ser limpa após 30 dias.
CONFIGURACAO_APP	versão mínima aceita do aplicativo	Compatibilidade de versões.

4. Novas tabelas no aparelho (somente locais)

Tabela	Conteúdo	Uso
META_SINCRONIZACAO	cursor de recebimento, data do último ciclo bem-sucedido	Retomar o recebimento de onde parou.
PACOTE_MAPA	identificador, região (limites), zoom mínimo e máximo, tamanho em bytes, data do download, unidades cobertas	Gestão dos pacotes offline (especificação 02).
REGISTRO_DIAGNOSTICO	data, tipo de evento, código, identificador do registro afetado	Diagnóstico de falhas, sem dados pessoais.

Nas tabelas locais UNIDADE_MANEJO e ATIVIDADE, acrescentar:

- `situacao`: SINCRONIZADO, PENDENTE, CONFLITO ou **REJEITADO** (novo estado; ver especificação 01, seção 3);
- `versao_base`;
- `id_operacao_pendente`;
- `motivo_rejeicao`.

5. Índices

Tabela	Índice	Motivo
UNIDADE_MANEJO, VERTICE, ATIVIDADE	`produtor_id` (primeira coluna do índice)	A documentação do Supabase recomenda indexar toda coluna usada em regras de acesso; sem índice, a leitura vira varredura completa.
VERTICE	(`unidade_id`, `ordem`), único	Contorno ordenado.
ATIVIDADE	`unidade_id`	Listar atividades da unidade.
REGISTRO_ALTERACAO	(`produtor_id`, número sequencial)	Recebimento paginado por produtor.

6. Regras de acesso por linha

Seguindo o padrão recomendado pela documentação do Supabase:

- uma regra por operação (leitura, inclusão, alteração), sempre aplicada ao papel de usuário autenticado;
- condição `produtor_id` igual ao usuário autenticado, com a função de usuário avaliada uma única vez por consulta e verificação explícita de usuário não nulo;
- as gravações do aplicativo passam pela função de envio (especificação 01). As tabelas podem ficar sem permissão de gravação direta para o papel autenticado, o que reduz a superfície de ataque.

7. Impacto nos diagramas da proposta

Ao aprovar estes ajustes, atualizar na proposta:

- **Modelo ER:** `produtor_id` em VERTICE e ATIVIDADE, `criado_em`, e as tabelas de servidor da seção 3;
- **Diagrama de classes:**
  - `SituacaoSincronizacao` passa a ter REJEITADO;
  - uma interface comum `RegistroSincronizavel` (`id`, `versao`, `versaoBase`, `situacao`) implementada por `UnidadeManejo` e `Atividade`;
  - as operações de sincronização passam a usar esse tipo em vez de `Object`;
- **Seção 7.4:** referenciar esta especificação e a 01.

8. Decisões funcionais que afetam o modelo (pendentes)

São opcionais, mas é melhor decidir **antes** de criar as tabelas, porque mudam colunas e a sincronização.

Decisão	Mudança no modelo, se aprovada
Exclusão de unidades e atividades	Coluna `excluido_em` (exclusão lógica), propagada pela sincronização como alteração comum.
Data e situação da atividade	Colunas `data_prevista` e `situacao_execucao` (PLANEJADA, EM_ANDAMENTO, CONCLUIDA).
Valores realizados	Colunas de custo realizado por item e receita realizada, ao lado dos previstos.
Perfis de equipe	Nova entidade de membros da propriedade e regras de acesso por propriedade, não só por produtor. É a mudança de maior impacto.